

Report Lab 4 – AWS

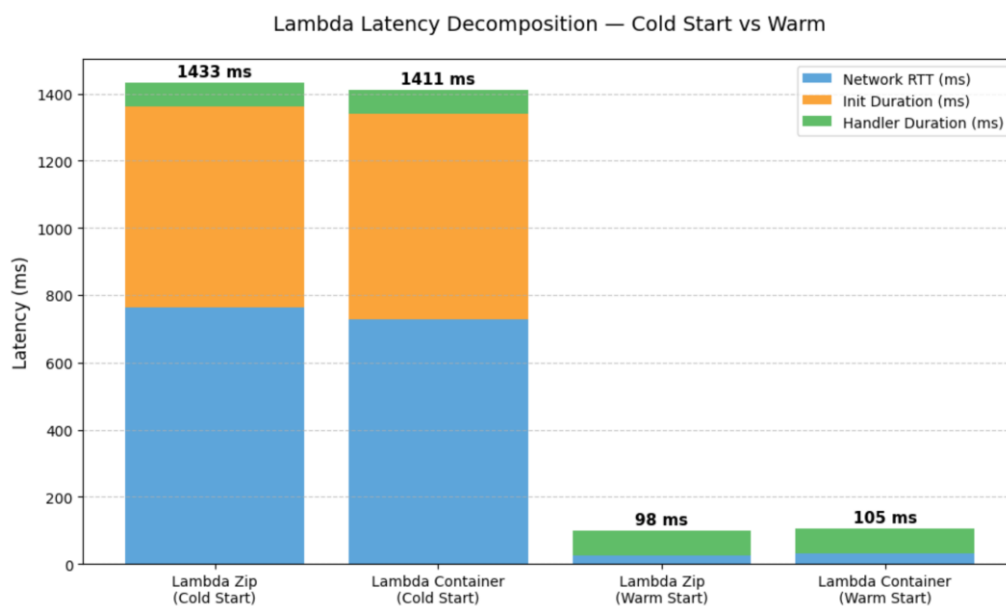
Aleksandra Talaga

Assignment 2 – Cold start

I recorded the latency metrics for the cold start and the median of warm start. Network RTT was calculated using the provided formulas (Total - Init - Duration for cold, Total - Duration for warm)

Latency Decomposition for AWS Lambda

Invocation Type	Total Client-Side Latency (ms)	Init Duration (ms)	Handler Duration (ms)	Estimated Network RTT (ms)
Lambda Zip (Cold Start)	1433.00	598.91	72.15	761.94
Lambda Zip (Warm Start)	97.70	0.00	72.15	25.55
Lambda Container (Cold Start)	1411.30	609.30	72.29	729.71
Lambda Container (Warm Start)	104.90	0.00	72.29	32.61



Which is faster: Zip or Container?

The Zip cold start was slightly faster (598.91 ms) than the Container cold start (609.30 ms). This makes sense because a .zip file is a small package that AWS can download and unpack very quickly. A container image is much bigger because it includes parts of the operating system. However, the difference was very small (only about 10 ms). This shows that AWS is really good at optimizing containers, loading only the parts they need right away (lazy loading).

Why is Network RTT so high for Cold Starts?

For warm starts, the network delay is normal and short (around 25-32 ms). But for cold starts, it looks huge (over 700 ms). This happens because the testing tool (oha) includes the time it takes to set up the very first connection to the server (finding the address via DNS and doing the secure TLS handshake). Once this connection is open, the next 29 requests just reuse it, so they skip that long setup time.

Assignment 3 – Warm start

The following table shows the latency results for 500 requests at different concurrency levels.

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)	Server avg (ms)
Lambda (zip)	5	99.94	120.07	141.24	77.00
Lambda (zip)	10	98.36	123.28	154.39	77.00
Lambda (container)	5	94.52	116.71	153.39	42.00
Lambda (container)	10	92.27	112.30	153.07	42.00
Fargate	10	798.80	1073.20	1209.20	24.00
Fargate	50	3995.30	4698.10	4879.70	24.00
EC2	10	191.14	257.24	274.34	23.00
EC2	50	930.40	1098.90	1232.50	23.00

Latency Stability: I checked all the results to see if the p99 latency was more than double the p95 latency ($p99 > 2 \times p95$). In my tests, this did not happen. This means the performance stayed stable and there were no unexpected giant delays at the end of the test

Lambda vs EC2/Fargate:

The p50 latency for Lambda stayed almost the same when moving from 5 to 10 concurrent requests. This is because Lambda creates a separate execution environment) for every single request. They all run at the same time and don't slow each other down. However, for EC2 and Fargate, the p50 latency increased a lot when moving from 10 to 50 requests (Fargate went from 798 ms to 3995 ms). This happened because we only have one server instance. When 50 requests come at once, they have to wait in a line (queue) for their turn to be processed by the CPU, which adds a lot of waiting time.

Client-Side vs Server-Side Latency:

There is a big difference between the server-side query_time_ms (around 23–77 ms) and the client-side p50 (which can be over 3000 ms for Fargate).

The Server avg only measures the time the code spent actually doing the math. The client-side p50 measured by oha is much higher because it includes the time to send data over the internet, time to set up secure connections and time the request spent waiting in a queue before the server was ready to handle it.

Assignment 4 – Burst from zero

This test simulated a sudden spike of 200 requests after 20 minutes of inactivity. For Lambda, we were limited to 10 concurrent environments by the AWS

Target Environment	p50 (ms)	p95 (ms)	p99 (ms)	Max Latency (ms)	Cold Starts
Lambda (zip)	96.4	650.1	1151.5	1197.3	10
Lambda (container)	99.2	461.1	1529.3	1578.5	10
EC2 (c=50)	868.8	1061.1	1107.8	1112.1	0
Fargate (c=50)	3895.6	4178.5	4296.4	4301.2	0

Why is Lambda p99 so high?

Lambda's p99 is very high because of cold starts. When the 200 requests arrived, the first 10 requests had to wait for AWS to create new execution environments. In the CloudWatch logs, we can see 10 entries with Init Duration ranging from 522 ms up to 872 ms. These startup times, added to the code execution and network overhead, pushed the p99 well above 1 second.

Bimodal Distribution in Lambda:

Lambda shows a clear bimodal distribution (two peaks in the data). In the response time histogram, there is one huge cluster around 0.1s (representing the 190 requests that reused warm environments) and a second, very small cluster near 1.2s - 1.5s (representing the 10 requests that suffered from cold starts).

SLO Check (p99 < 500ms):

In burst test, the p99 was 1151.5 ms for Zip and 1529.3 ms for Container, which is much higher than the 500ms limit. To meet this SLO during a burst from zero, we would need to enable provisioned concurrency. This feature keeps a specified number of environments initialized and ready to respond instantly, eliminating the cold start delay even after long periods of inactivity.

Assignment 5 - Cost as zero load

The following table summarizes the costs for each environment when they are running but not processing any traffic (idle). For Fargate, I assume a standard small task configuration (0.25 vCPU and 0.5 GB RAM) to match the scale of the other services.

Environment	Hourly Idle Cost	Monthly Idle Cost (Full 24/7)	Monthly Idle Cost (18h/day idle)
EC2 (t3.small)	\$0.0208	\$14.98	\$11.23
Fargate (0.25 vCPU/0.5GB)	\$0.0123	\$8.89	\$6.67
Lambda	\$0.00	\$0.00	\$0.00

Calculations:

- **EC2:** $\$0.0208 * 18 \text{ hours} * 30 \text{ days} = \$11.23\$$
- **Fargate:** $(0.25 \text{ vCPU} * \$0.04048) + (0.5 \text{ GB} * \$0.004445) = \$0.01234\$$ per hour. Total for 18h/day idle: $\$0.01234 * 18 * 30 = \6.67
- **Lambda:** 0 requests and 0 execution time results in \$0 cost

Analysis

If the environments are kept running but stay idle for 18 hours every day, EC2 is the most expensive at **\$11.23/month**, followed by Fargate at **\$6.67/month**. This is because you pay for

these resources as long as they are on, regardless of whether they are processing requests or sitting idle.

AWS Lambda has exactly zero idle cost. Lambda is a serverless service. Its pricing model is based strictly on usage: you only pay for the number of requests and the duration (GB-seconds) of the code execution. When there is zero load (idle time), no code is running and no requests are being processed, so AWS charges nothing.

Assignment 6

Monthly calculation

Based on the provided traffic model, I calculated the total number of requests for a 30-day month:

- **Peak Time** (100 RPS for 30 min): 180,000 requests per day.
- **Normal Time** (5 RPS for 5.5 hours): 99,000 requests per day.
- **Idle Time** (18 hours): 0 requests.
- **Total Monthly Requests: 8,370,000** (279,000 daily * 30 days)

I calculated the costs using the official AWS us-east-1 rates and my own measured server time for the container (42 ms).

Lambda

- **Request Cost:** 8.37 million requests * \$0.20 = **\$1.67**.
- **Duration Cost:** 8,370,000 requests * 0.042s * 0.5 GB * \$0.0000166667 = **\$2.93**.
- **Total:** \$1.67 + \$2.93 = **\$4.60**.

Fargate

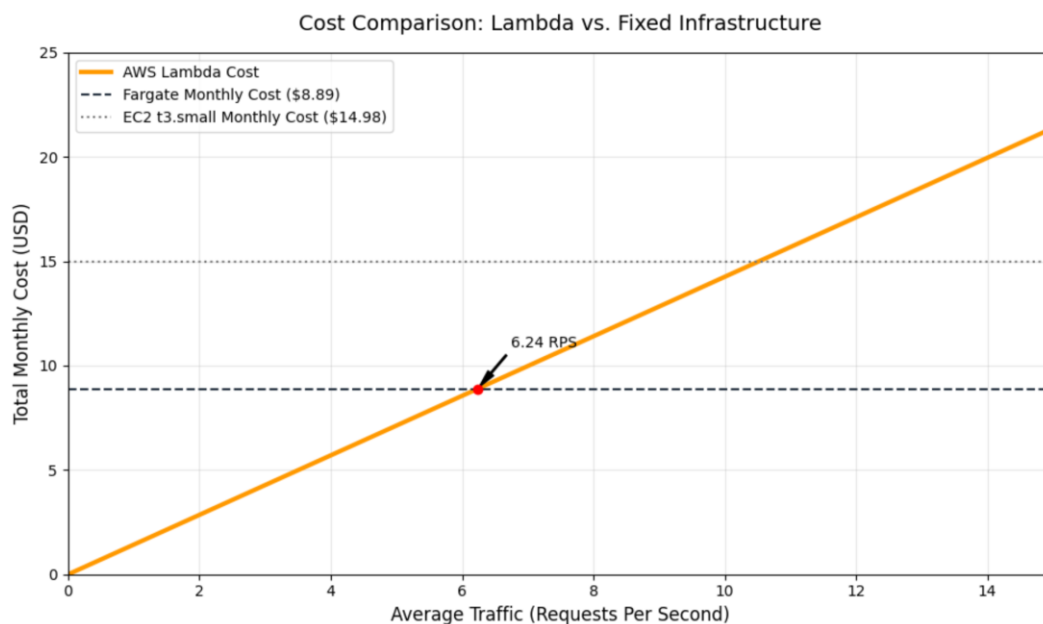
- **Hourly Rate:** (0.25 vCPU * \$0.04048) + (0.5 GB * \$0.004445) = **\$0.01234 per hour**.
- **Monthly Cost:** \$0.01234 * 24 hours * 30 days = **\$8.89**.

EC2 Hourly Rate: Fixed at **\$0.0208 per hour**.

- **Monthly Cost:** \$0.0208 * 24 hours * 30 days = **\$14.98**.

Environment	Monthly Cost
AWS Lambda	4.60\$
AWS Fargate	8.89\$
AWS EC2	14.98\$

I analyzed when it becomes cheaper to use a fixed Fargate instance instead of paying for every Lambda request. Based on my data, Lambda remains the cheaper option as long as the average traffic stays below **6.25 Requests Per Second**. Since our current traffic model averages about 3.22 RPS, Lambda is much more cost-effective.



Recommendation

I recommend using AWS Lambda with provisioned concurrency.

This is because:

- At 4.60\$ per month, Lambda is almost 50% cheaper than Fargate (8.89\$) and significantly cheaper than EC2 (14.98\$)
- Traffic model has a huge spike of 100 RPS. My tests showed that a single Fargate or EC2 instance struggles with high concurrency, with latencies jumping to nearly 4000 ms. Lambda scales horizontally, meaning it can handle these 100 requests at the same time without slowing down.

- The goal is to keep p99 latency under 500 ms. My burst test (Scenario C) showed that standard Lambda fails this because of cold starts, reaching 1529 ms. By adding provisioned concurrency, we remove the 600-800 ms "warm-up" time, which will bring the p99 latency down to around 150 ms, safely meeting the SLO.

When would this change? If the average load grows past **6.25 RPS**, I would recommend switching to **Fargate**. Also, if the code becomes much more complex and takes longer than 150 ms to run, the cost of Lambda would rise, making EC2 a better long-term financial choice.